# LeetCode Practice

## 2 Easy: Heap

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

**2**
**EASY**

**Patterns (in order)**

1. Heap (2)

**How to use**

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**2 questions | tap a title to jump to it**


**Easy questions (2)** Go >

**Heap (2)** Go >

E 1. Kth Largest Element in a Stream **LC 703**

E 2. Last Stone Weight **LC 1046**

## Easy questions (2)

EASY | TOPIC 1 OF 1

# Heap

**2 Easy questions start on the next page**

**Before this:** this is the first topic
**After this:** this is the last topic

---

**IN THIS TOPIC**

**1.** Kth Largest Element in a Stream LC 703
**2.** Last Stone Weight LC 1046

EASY | Heap | LeetCode #703 | 1 of 2 | 1/2

# Kth Largest Element in a Stream

## QUESTION

You are part of a university admissions office and need to keep track of the kth highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k, maintains a stream of test scores and continuously returns the kth highest test score after a new score has been submitted. More specifically, we are looking for the kth highest score in the sorted list of all scores.

Implement the KthLargest class:

### CONSTRAINTS

- KthLargest(int k, int[] nums) Initializes the object with the integer k and the stream of test scores nums.
- int add(int val) Adds a new test score val to the stream and returns the element representing the kth largest element in the pool of test scores so far.

**Example 1**

| | |
|---|---|
| **Input** | `["KthLargest", "add", "add", "add", "add", "add"] [[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]` |
| **Output** | `[null, 4, 5, 5, 8, 8]` |
| **Explanation** | *KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]); kthLargest.add(3); // return 4 kthLargest.add(5); // return 5 kthLargest.add(10); // return 5 kthLargest.add(9); // return 8 kthLargest.add(4); // return 8* |

**Example 2**

| | |
|---|---|
| **Input** | `["KthLargest", "add", "add", "add", "add"] [[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]` |
| **Output** | `[null, 7, 7, 7, 8]` |

- 0 <= nums.length <= $10^4$
- 1 <= k <= nums.length + 1
- $10^4$ <= nums[i] <= $10^4$
- $10^4$ <= val <= $10^4$
- At most $10^4$ calls will be made to add.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a class that maintains the kth largest element as numbers are continuously added via add(val). Right?"

"A few quick questions:

### PHASE 1 CLARIFY

Initial nums array may have fewer than k elements? Yes, but it's guaranteed there will be at least k elements after enough add() calls before we need a valid answer."

"Let me walk: k=3, nums=[4,5,8,2] -> add(3)->4, add(5)->5, add(10)->5, add(9)->8 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
def insert(x):
    append x to a list
def findLargest(k):
    sort the list in descending order and return
        the element at index k - 1
```

| TIME | SPACE |
|---|---|
| **O(n log n)** per query | **O(n)** |

WHY IT IS SLOW

Re-sorts every query and keeps every value. A min-heap of the k largest values (push, and pop if size exceeds k; the root is the answer) is O(log k) per add and O(k) space.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Min-heap of size k — O(log k) per add. Walk-through:"*

Init: heap = sorted k largest vals from nums (using a min-heap of size k).

Push all; pop until len == k.

`add(val): heap.push(val). If len(heap) > k: heap.pop(). Return heap[0] (min = k-th largest).`

**WHY min-heap of size k:** heap[0] = smallest of the k largest = k-th largest overall.

**WHY pop when size > k:** maintain exactly k elements.

**WHY heap[0] is the answer:** the minimum of the top-k is the k-th largest.

**Time:** O(log k) per add. Space: O(k).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _703_KthLargestElementInAStream:
    def __init__(self, k, nums):
        self.k = k
        self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)
```

```
    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

## PHASE 5 TEST & DEBUG

*"Let me trace how the heap property guides which element comes next."*

k=3, nums=[4,5,8,2]: heapify->[2,4,5,8]. trim to size 3: pop 2 -> heap=[4,5,8].

**add(3): push 3->[3,4,5,8], size4>3->pop min(3)->heap=[4,5,8]. return heap[0]=4 ok**

**add(5): push->[4,5,5,8], pop min(4)->[5,5,8]. return 5 ok**

The min-heap of size k always has the kth largest at its root after trimming

**WHY k-size min-heap:** heap minimum IS the kth largest at all times; add and evict maintains exactly k elements with O(log k) per insertion.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(log k)** per add, O(n log k) init | **O(k)** |

**Follow-up:** Kth Largest Element in an Array (#215) — one-shot version, quickselect is faster O(n) avg.

**Follow-up:** Find Median from Data Stream (#295) — two-heap technique for streaming median.

EASY | Heap | LeetCode #1046 | 2 of 2 | 2/2

# Last Stone Weight

## QUESTION

You are given an array of integers stones where stones[i] is the weight of the ith stone.

We are playing a game with the stones. On each turn, we choose the heaviest two stones and smash them together. Suppose the heaviest two stones have weights x and y with x <= y. The result of this smash is:

CONSTRAINTS

- If x == y, both stones are destroyed, and
- If x != y, the stone of weight x is destroyed, and the stone of weight y has new weight y - x.

At the end of the game, there is at most one stone left.

Return the weight of the last remaining stone. If there are no stones left, return 0.

**Example 1**

| | |
|---|---|
| Input | `stones = [2,7,4,1,8,1]` |
| Output | `1` |
| Explanation | *We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then,<br>we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then,<br>we combine 2 and 1 to get 1 so the array converts to [1,1,1] then,<br>we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of the last stone.* |

**Example 2**

| | |
|---|---|
| Input | `stones = [1]` |
| Output | `1` |

- 1 <= stones.length <= 30
- 1 <= stones[i] <= 1000

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Repeatedly smash the two heaviest stones together. If equal, both destroyed. If different,

the smaller is destroyed and the larger becomes (larger-smaller). Return the final remaining weight, or 0. Right?"

"A few quick questions:

At most one stone remains at the end? Yes."

"Let me walk: stones=[2,7,4,1,8,1] -> 1 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
while True:
    scan the array for the two heaviest stones,
        smash them, and put the remainder back
    if at most one stone is left:
        break
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) |

WHY IT IS SLOW

Scans the whole array to find the maxima every round. A max-heap pops the two heaviest in O(log n) each: O(n log n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Max-heap simulation — O(n log n). Walk-through:"*

Push all stones into a max-heap (negate for Python's min-heap).

While heap has 2+ stones:

`y = -heappop(heap) <- heaviest stone.`

`x = -heappop(heap) <- second heaviest.`

If y != x: heappush(heap, -(y - x)) <- remainder stone.

<- If y == x: both destroyed, nothing pushed.

Return -heap[0] if heap else 0.

**WHY max-heap:** always smash the two heaviest stones (problem rule).

Each smash reduces the stone count by at least 1 (sometimes 2 if equal) -> terminates.

**Time:** O(n log n). Space: O(n).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _1046_LastStoneWeight:
    def lastStoneWeight(self, stones):
        heap = [-s for s in stones]
        heapq.heapify(heap)
        while len(heap) > 1:
            first = -heapq.heappop(heap)
            second = -heapq.heappop(heap)
            if first != second:
                heapq.heappush(heap, -(first - second))
        return -heap[0] if heap else 0
```

## PHASE 5 TEST & DEBUG

*"Let me trace how the heap property guides which element comes next."*

stones=[2,7,4,1,8,1]: heap=[-8,-7,-4,-2,-1,-1]. pop 8,7->diff=1->push -1. heap has [-4,-2,-1,-1,-1].

pop 4,2->diff=2->push -2. heap=[-2,-1,-1,-1]. pop 2,1->diff=1->push -1. heap=[-1,-1,-1].

pop 1,1->equal->nothing pushed. heap=[-1]. -> 1 ok

Negating values turns Python's min-heap into a max-heap for O(log n) heaviest-stone access

**WHY max-heap:** always smashing the two heaviest stones first is greedy-optimal — heap gives the two heaviest in O(log n) each round.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n log n)** | **O(n)** |

**Follow-up:** Last Stone Weight II (#1049) — partition into two groups to minimize difference, needs DP not greedy.

**Follow-up:** K Closest Points to Origin (#973) — similar heap-based selection pattern.